

Tecnológico de Monterrey

Evidencia 2 – Estructuras de Datos Lineales

Programación de estructuras de datos y algoritmos fundamentales | TC1031.607

David Alonso Cantú Delgado.

Regina Mugica Limon | A00843542

18 de Octubre del 2026
Monterrey, Nuevo León

Pruebas

Parte A

Para comprobar el funcionamiento de las estructuras de datos utilizadas en el proyecto, se realizaron pruebas individuales de LinkedList, Queue, Stack y DoublyLinkedList. Se probaron estructuras vacías, inserciones, eliminaciones, consulta del tamaño y recorridos. También se comprobó el comportamiento de Queue y Stack al intentar eliminar un elemento cuando se encuentran vacías.



Figura 1. *Pruebas de funcionamiento de LinkedList, Queue, Stack y DoublyLinkedList.*

Carga de registros

Para la lectura de los datos se utilizó una Queue como buffer, permitiendo procesar los registros en el mismo orden en el que fueron leídos. En la prueba con `log607-1.txt` se cargaron correctamente 6818 registros y no se omitió ningún registro.

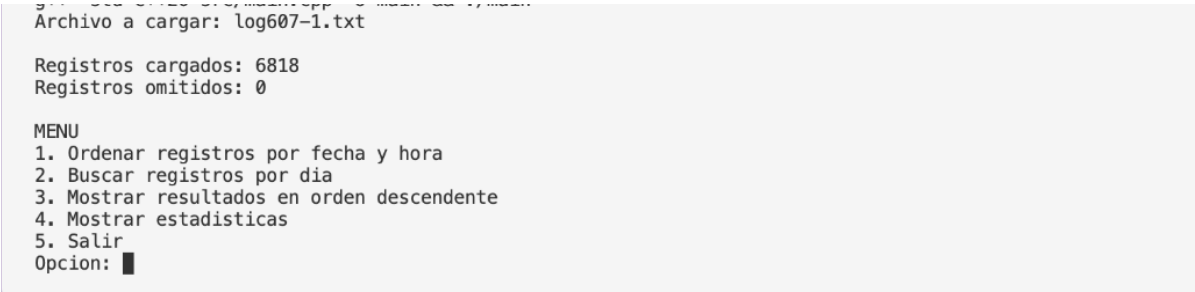


Figura 2. *Carga de los registros y menú principal del programa.*

Ordenamiento de log607-1.txt

Se utilizó Merge Sort para ordenar los registros por fecha y hora. Antes de ejecutar el algoritmo se predijo una complejidad de O(n log n). El programa registra el tiempo de ejecución, número de comparaciones y profundidad máxima de recursión.

```
Archivo a cargar: log607-1.txt

Registros cargados: 6818
Registros omitidos: 0

MENU
1. Ordenar registros por fecha y hora
2. Buscar registros por dia
3. Mostrar resultados en orden descendente
4. Mostrar estadisticas
5. Salir
Opcion: 1

Antes de ordenar, cual es tu prediccion de complejidad?
1. O(n log n)
2. O(n^2)
Prediccion: 1

Ordenando registros...
Archivo ordenado guardado en out/output607.out

Algoritmo: Merge Sort
Archivo: log607-1.txt
Registros: 6818
Tiempo: 23726 microsegundos
Comparaciones: 78330
Profundidad maxima: 14
La prediccion fue correcta: Merge Sort es O(n log n).
```

Figura 3. *Resultados del ordenamiento con Merge Sort para log607-1.txt.*

Búsqueda por día

Después de ordenar los registros, se realizó una búsqueda correspondiente al día Sep 14 2024. El programa encontró un total de 12 registros correspondientes a esta fecha.

```
MENU
1. Ordenar registros por fecha y hora
2. Buscar registros por dia
3. Mostrar resultados en orden descendente
4. Mostrar estadisticas
5. Salir
Opcion: 2
Ingresa el dia a buscar (ejemplo: Sep 14 2024): Sep 14 2024
Sep 14 2024 00:24:23 10.14.115.26 Social engineering attacks
Sep 14 2024 00:25:20 10.14.177.24 Suspicious IP Addresses
Sep 14 2024 00:52:28 10.14.175.238 Security breaches
Sep 14 2024 01:18:02 10.14.66.138 Anomalous User Activity
Sep 14 2024 05:14:25 10.14.85.112 Data Exfiltration Attempts
Sep 14 2024 05:15:49 10.14.58.71 Port Scanning Activity
Sep 14 2024 11:57:05 10.14.197.208 Cross-Site Scripting
Sep 14 2024 18:34:13 10.14.65.141 Anomalous User Activity
Sep 14 2024 20:00:03 10.14.47.235 Data Exfiltration Attempts
Sep 14 2024 22:23:31 10.14.141.44 Failed Login Attempts
Sep 14 2024 23:03:07 10.14.175.98 Port Scanning Activity
Sep 14 2024 23:30:59 10.14.41.52 Suspicious IP Addresses
Total encontrados: 12

MENU
```

Figura 4. *Búsqueda de registros correspondientes al 14 de septiembre de 2024.*

Resultados en orden descendente

Los registros obtenidos en la búsqueda se almacenaron en un Stack. Debido a su funcionamiento LIFO, los registros se muestran desde el más reciente hasta el más antiguo.

```
MENU
1. Ordenar registros por fecha y hora
2. Buscar registros por dia
3. Mostrar resultados en orden descendente
4. Mostrar estadisticas
5. Salir
Opcion: 3

Registros en orden descendente:
Sep 14 2024 23:30:59 10.14.41.52 Suspicious IP Addresses
Sep 14 2024 23:03:07 10.14.175.98 Port Scanning Activity
Sep 14 2024 22:23:31 10.14.141.44 Failed Login Attempts
Sep 14 2024 20:00:03 10.14.47.235 Data Exfiltration Attempts
Sep 14 2024 18:34:13 10.14.65.141 Anomalous User Activity
Sep 14 2024 11:57:05 10.14.197.208 Cross-Site Scripting
Sep 14 2024 05:15:49 10.14.58.71 Port Scanning Activity
Sep 14 2024 05:14:25 10.14.85.112 Data Exfiltration Attempts
Sep 14 2024 01:18:02 10.14.66.138 Anomalous User Activity
Sep 14 2024 00:52:28 10.14.175.238 Security breaches
Sep 14 2024 00:25:20 10.14.177.24 Suspicious IP Addresses
Sep 14 2024 00:24:23 10.14.115.26 Social engineering attacks
```

Figura 5. *Uso de Stack para mostrar los registros encontrados en orden descendente.*

Estadísticas de los registros

Finalmente, el programa obtiene estadísticas generales de los registros procesados. Se muestra el total de registros, el día con mayor cantidad de incidentes, el tipo de evento más frecuente y las direcciones IP con mayor frecuencia. En esta ejecución se analizaron 6818 registros.

```
MENU
1. Ordenar registros por fecha y hora
2. Buscar registros por dia
3. Mostrar resultados en orden descendente
4. Mostrar estadisticas
5. Salir
Opcion: 4

ESTADISTICAS
Total de registros: 6818
Dia(s) con mas incidentes:
Jan 27 2025: 21
Evento(s) mas frecuente(s):
Malware Detection Alerts: 342
IP(s) mas frecuente(s):
10.14.199.224: 3
10.14.200.76: 3
10.14.161.42: 3
10.14.29.229: 3
10.14.186.139: 3
10.14.36.7: 3
10.14.242.186: 3
10.14.162.205: 3
10.14.176.221: 3
10.14.126.198: 3
10.14.94.165: 3
10.14.69.71: 3
10.14.123.164: 3
```

Figura 6. *Estadísticas obtenidas a partir de los registros procesados.*

Ordenamiento de log607-2.txt

Se realizó la misma prueba de ordenamiento con el segundo archivo para comparar el comportamiento de Merge Sort utilizando la misma cantidad de registros. El archivo contenía 6818 registros y nuevamente se obtuvo una profundidad máxima de recursión de 14.

```
Archivo a cargar: log607-2.txt

Registros cargados: 6818
Registros omitidos: 0

MENU
1. Ordenar registros por fecha y hora
2. Buscar registros por dia
3. Mostrar resultados en orden descendente
4. Mostrar estadisticas
5. Salir
Opcion: 1

Antes de ordenar, cual es tu prediccion de complejidad?
1. O(n log n)
2. O(n^2)
Prediccion: 1

Ordenando registros...
Archivo ordenado guardado en out/output607.out

Algoritmo: Merge Sort
Archivo: log607-2.txt
Registros: 6818
Tiempo: 14724 microsegundos
Comparaciones: 50624
Profundidad maxima: 14
La prediccion fue correcta: Merge Sort es O(n log n).

MENU
1. Ordenar registros por fecha y hora
2. Buscar registros por dia
3. Mostrar resultados en orden descendente
4. Mostrar estadisticas
5. Salir
Opcion:
```

Figura 7. *Resultados del ordenamiento con Merge Sort para log607-2.txt.*

Comparación de resultados del ordenamiento

Archivo	***Registros***	***Tiempo (µs)***	***Comparaciones***	***Profundidad máxima***
log607-1.txt	*6818*	*23,726*	*78,330*	*14*
log607-2.txt	*6818*	*14,724*	*50,624*	*14*

Ambos archivos contienen la misma cantidad de registros y alcanzaron una profundidad máxima de recursión de 14. Sin embargo, log607-2.txt realizó menos comparaciones que log607-1.txt. Esto muestra que, aunque Merge Sort mantiene una complejidad de O(n log n), el número exacto de comparaciones y el tiempo de ejecución pueden variar dependiendo de los datos procesados y de cada ejecución.

Casos especiales

Se realizaron pruebas adicionales para comprobar el comportamiento del programa ante diferentes situaciones. Se probó la carga de un archivo inexistente, el ingreso de una opción inválida en el menú y el intento de realizar una búsqueda antes de ordenar los registros. En cada caso, el programa mostró un mensaje indicando la situación y evitó continuar con una operación incorrecta.

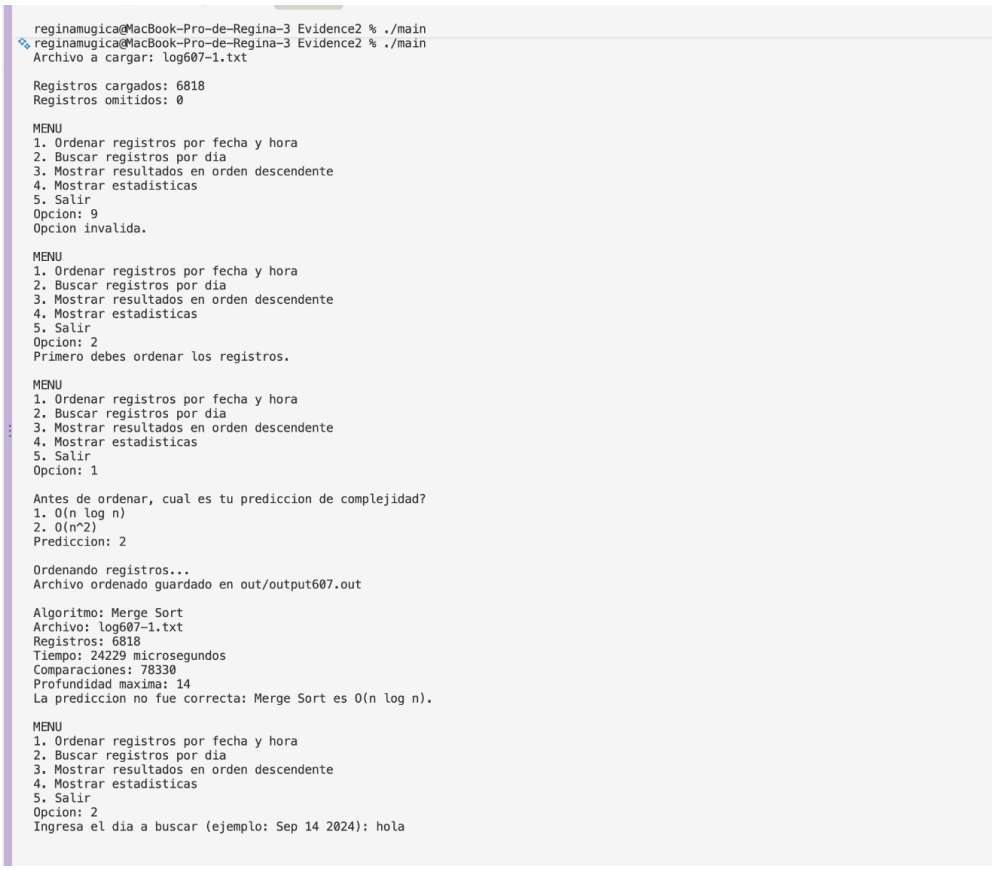


Figura 8. *Pruebas de casos especiales y validaciones del programa.*

Uso de Inteligencia Artificial

Durante el desarrollo de la evidencia utilicé Inteligencia Artificial como apoyo para comprender la implementación de las estructuras de datos y adaptar algunos algoritmos a las clases desarrolladas durante el curso. Las sugerencias obtenidas fueron revisadas y probadas antes de integrarlas al programa.

Interacción 1: Implementación de Merge Sort

Prompt / pregunta realizada:

> ¿Cómo puedo adaptar Merge Sort para ordenar los registros directamente en una lista doblemente ligada sin utilizar std::sort y contando las comparaciones y la profundidad de recursión?

Respuesta o sugerencia de la IA:

La IA sugirió dividir la lista en dos partes utilizando apuntadores y posteriormente combinar ambas partes de forma ordenada. También sugirió utilizar variables para contar las comparaciones realizadas y registrar la profundidad alcanzada por las llamadas recursivas.

¿Cómo se utilizó o modificó?

La propuesta se adaptó a la clase DoublyLinkedList utilizada durante el curso. Se implementaron las funciones necesarias para dividir y unir los nodos, manteniendo las conexiones next y prev. Después se realizaron pruebas con ambos archivos para comprobar el funcionamiento del algoritmo.

Interacción 2: Uso de Queue y Stack

Prompt / pregunta realizada:

> ¿Cómo puedo utilizar Queue para procesar los registros del archivo y Stack para mostrar los resultados de una búsqueda en orden descendente?

Respuesta o sugerencia de la IA:

La IA sugirió utilizar Queue como un buffer de lectura para conservar el orden de llegada de las líneas del archivo. Para los resultados de búsqueda, sugirió insertar los registros encontrados en un Stack para aprovechar su comportamiento LIFO.

¿Cómo se utilizó o modificó?

Se utilizaron las estructuras implementadas en el proyecto y no las estructuras de la biblioteca estándar de C++. Queue procesa los registros en el mismo orden en que fueron leídos, mientras que Stack permite mostrar primero el registro más reciente y terminar con el más antiguo.

Interacción 3: Error de la IA y corrección

Prompt / situación:

> ¿Cómo puedo probar los recorridos y el tamaño de mi DoublyLinkedList en las pruebas de la Parte A?

Problema con la respuesta de la IA:

Una de las modificaciones sugeridas provocó que algunos métodos de DoublyLinkedList, como getSize, printForward, printBackward y getHead, quedaran declarados o definidos más de una vez. Al compilar, aparecieron errores como class member cannot be redeclared y redefinition.

¿Cómo se corrigió?

Se revisó el archivo DoublyLinkedList.h, se identificaron las declaraciones y definiciones repetidas y se eliminaron los duplicados. Después de realizar la corrección, se volvió a compilar ParteA.cpp y las pruebas se ejecutaron correctamente para LinkedList, Queue, Stack y DoublyLinkedList.

Reflexión sobre el uso de IA

> El uso de Inteligencia Artificial fue útil como apoyo durante el desarrollo, principalmente para comprender cómo adaptar los algoritmos a las estructuras del proyecto y encontrar posibles soluciones a errores. Sin embargo, también fue necesario revisar y probar las respuestas, ya que no todas las sugerencias funcionaron correctamente desde el primer intento. Esto permitió utilizar la IA como una herramienta de apoyo y no simplemente copiar sus respuestas.

Reflexión final

Durante esta evidencia pude aplicar de una forma más completa las diferentes estructuras de datos lineales que vimos durante el curso. En lugar de utilizarlas de manera separada, cada una tuvo una función específica dentro del programa para poder leer, almacenar, ordenar, buscar y analizar los registros de eventos de red.

La LinkedList funcionó como la estructura base para implementar Queue y Stack. Al manejar apuntadores al inicio y al final de la lista, fue posible realizar operaciones como agregar elementos al final o eliminar el primero sin tener que recorrer toda la estructura. Esto fue importante porque estas operaciones se realizan muchas veces durante el procesamiento de los registros.

La Queue se utilizó durante la lectura de los archivos. Su funcionamiento FIFO (First In, First Out) permite que el primer registro que entra sea también el primero que sale. Esto ayudó a conservar el orden original de llegada de los datos antes de almacenarlos en la lista doblemente ligada. En las pruebas realizadas, ambos archivos permitieron cargar correctamente 6818 registros sin omitir ninguno.

La DoublyLinkedList fue la estructura principal para almacenar los registros. Al tener apuntadores next y prev, permite recorrer los elementos en ambas direcciones. También fue sobre esta estructura donde se implementó Merge Sort para ordenar los registros de acuerdo con su

fecha y hora. Aunque una lista doblemente ligada ya permite realizar un recorrido hacia atrás, se utilizó también un Stack para cumplir una función específica dentro del programa y aplicar otra de las estructuras estudiadas.

El Stack se utilizó después de realizar una búsqueda por día. Los registros encontrados se agregan al Stack y, debido a su funcionamiento LIFO (Last In, First Out), pueden mostrarse comenzando por el registro más reciente y terminando con el más antiguo. Por ejemplo, en la búsqueda del 14 de septiembre de 2024 se encontraron 12 registros, los cuales pudieron mostrarse posteriormente en orden descendente.

Para el ordenamiento elegí Merge Sort, ya que su complejidad esperada es O(n log n) y puede adaptarse a una lista ligada mediante el manejo de nodos. En las pruebas, ambos archivos tuvieron 6818 registros y una profundidad máxima de recursión de 14. A pesar de tener el mismo número de elementos, el número de comparaciones y el tiempo de ejecución fueron diferentes. Esto demuestra que las mediciones reales pueden variar dependiendo de los datos y de la ejecución, aunque la complejidad del algoritmo siga siendo O(n log n). La profundidad obtenida también tiene sentido debido a que Merge Sort divide repetidamente el conjunto de datos en partes más pequeñas.

Otra decisión importante fue no utilizar búsqueda binaria directamente sobre la lista ligada. En un arreglo se puede acceder directamente a una posición utilizando su índice, pero en una lista ligada es necesario recorrer los nodos para llegar a una posición determinada. Por esta razón, buscar repetidamente el elemento de en medio eliminaría gran parte de la ventaja de la búsqueda binaria. En este programa se aprovecha que los registros ya están ordenados para recorrerlos secuencialmente y detener la búsqueda cuando la fecha ya superó el día solicitado.

Finalmente, esta evidencia me ayudó a entender mejor que no existe una sola estructura adecuada para todos los problemas. Queue fue útil para conservar el orden de llegada, DoublyLinkedList para almacenar y recorrer los registros, Stack para invertir el orden de los resultados y LinkedList como base para otras estructuras. También pude comprobar la importancia de analizar y probar las soluciones propuestas por Inteligencia Artificial, ya que algunas sugerencias tuvieron que ser corregidas antes de funcionar correctamente. Más que utilizar las estructuras solamente porque eran parte de la actividad, el proyecto me permitió entender para qué situación puede ser más conveniente utilizar cada una.